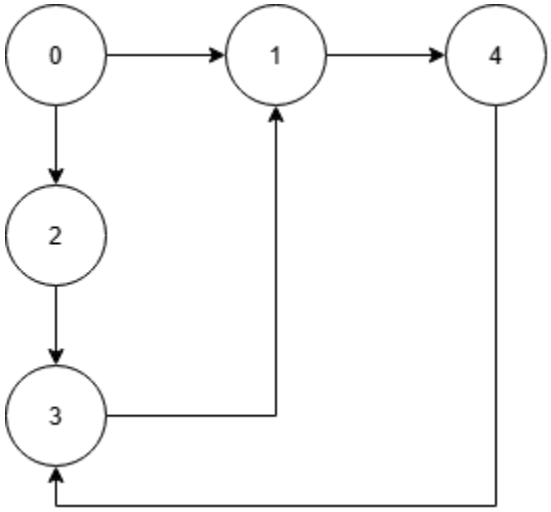
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	October 2, 2026	**DATE SUBMITTED:**	October 2, 2026
NAME:	Silao, Johrel Louie T.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _10.1_ Graphs

1. PROCEDURE OUTPUT

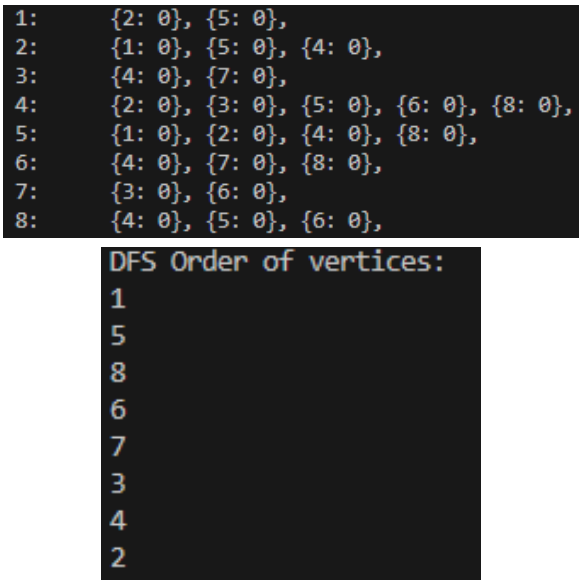
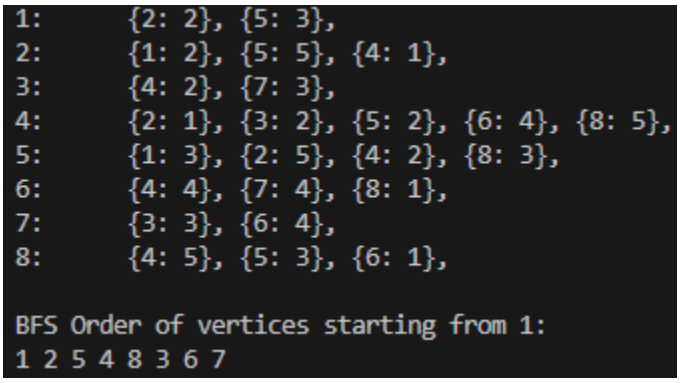
ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list



ILO A.1

Output	Graph adjacency list (start_vertex, end_vertex, weight): (0, 2, 4) (0, 1, 2) (1, 4, 3) (2, 3, 2) (3, 1, 4) (4, 3, 3)

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

ILO B.1 Output	
ILO B.2 Output	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

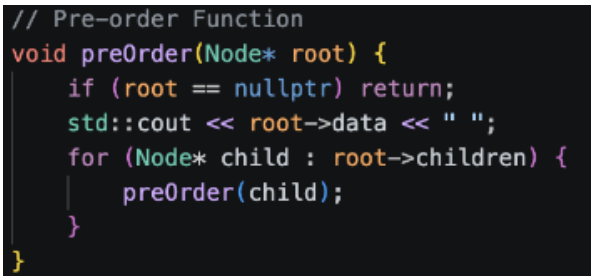
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.**

    - Depth-First Search (DFS) would be the most helpful algorithm because it will first explore all possible paths before backtracking to previous vertices.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**

    - Among the three tree traversals, Pre-Order traversal is the closest to the DFS because it first visits the root node before exploring through different paths in the subtree.

Pre-Order Tree Traversal	Depth-First Search
	

```
// Pre-order Function
void preOrder(Node* root) {
    if (root == nullptr) return;
    std::cout << root->data << " ";
    for (Node* child : root->children) {
        preOrder(child);
    }
}
```

```
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

3. **In the performed code, what data structure is used to implement the Breadth First Search?**

    - Queue is the data structure that is used to implement the Breadth First Search.

4. **How many times can a node be visited in the BFS?**

    - In a BFS, a node can be visited only once, however it can be checked using the adjacency lists numerous times.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.